

ENSAE 1A – Projet de programmation 2026

Responsable du cours: Patrick Loiseau



Version du 26 janvier 2026

Note : Ce document présente la partie “accélérée” du projet de programmation. Pour les élèves suivant le cours “projet de programmation accéléré”, il s’agit de l’ensemble du cours. Les élèves qui suivent le cours “projet de programmation lent” commenceront par 4 séances de mini-projet beaucoup plus guidé (façon TP) puis enchaîneront sur le travail décrit dans ce document à partir de la 5ème séance de 1h30.

Toutes les ressources associées au projet pour les étudiants (base de code, fichiers d’input, tests préliminaires, etc.) se trouve sur le dépôt github (publique) suivant :

<https://github.com/patrickloiseau/ensae-prog26>.

Table des matières

1	Instructions et recommandations générales	2
1.1	Objectifs	2
1.2	Environnement	2
1.3	Règles spécifiques : binômes et utilisation d’IA générative	2
1.4	Évaluation et rendus	3
1.5	Ressources utiles	3
1.6	Contact des chargés de TP	3
2	Description du problème et du code fourni	5
2.1	Description général du problème	5
2.2	Tâches	5
2.2.1	Partie 1 : Chemin de temps minimal	5
2.2.2	Partie 2 : Améliorations algorithmiques	6
2.2.3	Partie 3 : Extensions	6
2.3	Organisation indicative des séances	7
2.4	Description du code fourni	7
3	Quelques rappels d’environnement	9
3.1	Créer un dépôt git sur GitHub avec les fichiers du TP	9
3.2	Résumé sur l’utilisation du terminal	10
3.3	Classes	11
3.4	Tests unitaires	12

1 Instructions et recommandations générales

1.1 Objectifs

Le projet de programmation a pour objectif global de faire acquérir une expérience de programmation par la pratique, via la résolution d'un problème particulier (décrit dans la section suivante). Plus spécifiquement, il est attendu au cours du projet que vous acqueriez un certain nombre de compétences incluant :

- Apprendre à analyser un problème et à le traduire en terme algorithmique
- Apprendre à comprendre une base de code donnée et à s'y adapter
- Apprendre à structurer du code pour un projet de moyenne envergure
- Apprendre à chercher en ligne les informations, les documentations, et les librairies utiles
- Apprendre à analyser la complexité d'un algorithme
- Apprendre à manipuler quelques algorithmes et structures de données classiques
- Apprendre à debugger, tester, et optimiser du code
- Se familiariser avec les bonnes pratiques de code et avec un environnement d'édition.

L'objectif du projet de programmation est purement **pédagogique**. Il ne s'agit pas de produire un code destiné à être utilisé par la suite en production, mais **d'apprendre** les compétences mentionnées ci-dessus car elles sont utiles dans la vie professionnelle.

Par ailleurs, le projet vise des compétences générales en programmation. Ainsi, bien que la plupart des étudiants soient amenés à utiliser la programmation dans le cadre de la data science, ce projet ne contient volontairement aucun élément de data science. Ces éléments feront l'objet de cours spécifiques en 2A et 3A.

1.2 Environnement

Le projet est fait en **Python** (obligatoirement). On utilisera la notion de classe, avec une structure minimale imposée (voir plus loin), et on utilisera des tests unitaires. Pour l'ensemble du projet, on demande l'utilisation de l'éditeur **vscode** (il n'est **pas autorisé d'utiliser des notebook jupyter**). Il peut être utilisé soit sur un serveur cloud type onyxia (recommandé) soit en local sur les machines des salles informatiques. Il est également possible d'utiliser vscode sur une machine personnel, mais dans ce cas les élèves doivent pouvoir installer de façon autonome ce dont ils ont besoin.

Cette année, on recommande l'utilisation de la plateforme Onyxia de l'INSEE. Pour y accéder, allez sur <https://datalab.sspcloud.fr/> et sélectionnez **vscode-python**. Il vous faudra créer un compte si vous n'en avez pas déjà un ; utilisez pour cela votre adresse email ensae. On impose également de mettre le projet dans un dépôt git privé (ajouter votre chargé de TP dont les coordonnées sont données plus loin). La prise en main de cet environnement sera l'objet de la première séance. Une documentation sur le datalab peut également être trouvée sur pamplemousse (fichier `HowToDatalab.pdf`).

Note : Le service sur la plateforme Onyxia permet d'accéder à un environnement uniforme avec des logiciels à jour de n'importe où, y compris à la maison. Attention toutefois à bien enregistrer votre travail (sur git) car un service inutilisé peut être "tué" à tout moment et le travail serait alors perdu.

1.3 Règles spécifiques : binômes et utilisation d'IA générative

Le projet est fait en binôme. **Les deux membres d'un binôme doivent appartenir au même groupe de TP, et il n'est pas possible de changer de groupe de TP.** Les binômes sont à constituer et à indiquer à votre chargé de TP lors de la séance 1. La présence aux TPs est obligatoire et sera vérifiée au début de chaque séance. Des pénalités seront appliqués pour les absences non justifiées.

L'objectif du cours est d'acquérir une compétence en programmation. Celle-ci est essentielle pour le reste des études et pour la vie professionnelle de la plupart d'entre vous. En effet, les IA génératives peuvent aider à augmenter (significativement) la vitesse à laquelle un codeur expérimenté code ; mais il est crucial pour utiliser cette aide efficacement de pouvoir bien *comprendre* les sorties de l'IA générative et cette compréhension ne peut être acquise que par la pratique.¹ En conséquence, dans le projet de programmation, on tolère l'utilisation de l'IA à des fins de documentation, par exemple pour chercher des conseils généraux sur un algorithme, une structure

1. Les développeurs qui aujourd'hui utilise efficacement l'IA ont d'ailleurs appris à une époque où elle n'existait pas.

de donnée, ou autre. Mais il est **strictement interdit d'utiliser une IA pour générer du code. Ceci est sanctionné par le règlement de l'école (tout comme le plagiat)**. Les chargés de TP auront pour vocation de tester de façon détaillée la connaissance et la compréhension par chacun des deux membres du binôme du code produit et de la façon dont il a été produit. Ils pourront le faire par des questions précises et ciblées, en demandant de coder ou re-coder devant eux une fonction ou d'en modifier une, ou par tout autre moyen approprié. La compréhension ainsi testée aura un poids déterminant dans la notation.

1.4 Évaluation et rendus

L'évaluation se fera en 2 étapes :

Un rendu intermédiaire, dû au démarrage de la séance 6 (le 9 mars à 8h30) : on demande le rendu de l'ensemble du code écrit jusque-là, testé et documenté. Le code doit être déposé sur le git privé du binôme (auquel vous aurez ajouté votre chargé de TP). Il n'est pas demandé de rapport à ce stade.

Un rendu final, dû le 14 avril (23h59) : on demande le rendu final de l'ensemble du code du projet, documenté, accompagné d'un rapport d'environ 4-6 pages au format PDF. Le rapport doit être concis, clair et précis, et décrire succinctement les choix algorithmiques et les résultats et justifier la complexité des algorithmes utilisés. Le code et le rapport PDF doivent être déposés sur le git privé du binôme

Lors de la dernière séance, chaque chargé de TP passera voir tous les binômes de son groupe pour une séance de questions destinée à évaluer la compréhension des deux membres du binôme (cf. section précédente). Il ne sera pas demandé de faire une présentation du projet (pas de slide), mais il faudra pouvoir faire une démo si demandé.

La notation inclura 3 points (sur 20) pour le rendu intermédiaire. Nous serons particulièrement attentifs au fait que le code soit testé et bien documenté. Le reste (17 points) sera attribué sur la base du rendu final et de la séance de questions lors du dernier TP. Nous tiendrons compte d'un certain nombre de critères : qualité du code, clarté de la documentation, pertinence des choix algorithmiques, analyse des performances algorithmiques, respect des bonnes pratiques, etc. Il est important de noter que la qualité du code (et de sa documentation), et surtout la compréhension par les deux membres du binôme, est un critère essentiel dans la notion, bien plus encore que la quantité de code produite.

1.5 Ressources utiles

Le projet suppose une connaissance du langage Python acquise au premier semestre. Toutefois, en cas de besoin, il existe de nombreuses ressources en ligne et de nombreux cours et livres sur la programmation en Python, tels que *Introduction to programming in Python* de R. Sedgewick, K Wayne, et R. Dondero. Pour ce qui est des bonnes pratiques, il existe de nombreuses ressources. Citons en particulier :

- Les documents de la communauté Python, e.g., le [PEP8](#) (qui contient une section sur le nommage) et le [PEP257](#) (sur l'écriture de documentation)
- Le [Hitchhiker's Guide to Python](#) et le livre [clean code](#).
- Les supports de Lino Galiana et Romain Avouac (enseignant des cours de deuxième et troisième année) sur [la qualité du code](#) et sur [git](#).

Il pourra être utile de s'y référer au long du projet. Enfin, les élèves pourront être amenés à vouloir consulter des livres d'algorithmique ; il en existe là aussi de nombreux (e.g., *Introduction to Algorithms* de Cormen et al., *Algorithm Design* de Kleinberg et Tardos, ou *Algorithms* de Dasgupta, Papadimitriou, et Vazirani).

1.6 Contact des chargés de TP

- Groupe 1 : Patrick Loiseau,
- Groupe 2 : Simon Mauras,
- Groupe 3 : Guillaume Lev,
- Groupe 4 : Idrissa Niakh,
- Groupe 5 : Melissa Tamine,
- Groupe 6 : Patrick Loiseau,
- Groupe 7 : Simon Mauras,
- Groupe 8 : Guillaume Levy,

— Groupe 9 : Marius Potfer,

— Groupe 10 : Richard Ahouansou,

2 Description du problème et du code fourni

2.1 Description général du problème

On considère un environnement constitué d'un certain nombre (fini) de points dans l'espace. L'objectif global sera de déplacer un agent d'un point initial à un point final. Il existe des routes entre certains couples de points. Ces routes ont deux attributs : une longueur et coefficient d'augmentation de fatigue lié à la qualité du terrain (par exemple, parcourir une route marécageuse augmente plus la fatigue de l'agent que de parcourir une route bétonnée). La fatigue de l'agent impacte sa vitesse, et donc le temps de parcours pour une longueur donnée.

Formellement, on représente l'environnement comme un graphe dont les sommets $V = \{1, \dots, n\}$ sont les points de l'environnement. Les arêtes correspondent aux chemins : il existe une arête (i, j) s'il y a un chemin entre le point i et le point j . Les arêtes sont orientées et chaque arête $e = (i, j)$ a deux attributs qui sont des entiers :

- une longueur $l(e) \geq 1$;
- un coefficient d'augmentation de fatigue $f(e) \geq 0$.

On note E l'ensemble des arêtes et $m = |E|$ le nombre d'arêtes (sachant que n représente le nombre de sommets).

Étant donné ce graphe, le temps total de parcours d'un chemin $(v_0, \dots, v_k)$ contenant les k arêtes $e_1 = (v_0, v_1), \dots, e_k = (v_{k-1}, v_k)$ est défini comme suit : on considère que l'agent démarre au sommet v_0 avec une fatigue f égale à 1. Chaque sommet est traversé avec une vitesse égale à l'inverse de la fatigue. A la fin de l'arête e la fatigue augmente d'un facteur $f(e)$. Ainsi on a

$$\text{temps}(v_0, \dots, v_k) = l(v_0, v_1) + l(v_1, v_2) \times (1 + f(v_0, v_1)) + \dots + l(v_{k-1}, v_k) \times (1 + f(v_0, v_1) + f(v_{k-2}, v_{k-1})).$$

Le but principal du jeu sera de trouver, étant donnés un point initial v_s et un point final v_t , un chemin de temps minimum allant de v_s à v_t . Cela correspond à une "mission" (amener un agent d'un point v_s à un point v_t). On considèrera ensuite des variantes où l'agent a plusieurs missions, où il y a plusieurs agents, etc.

*Note : dans le code, l'environnement ci-dessus où les arêtes ont une longueur et un coefficient d'augmentation de fatigue sera nommé **network** et non pas **graph**, on réservera le nom **graph** pour des graphes standards n'ayant qu'un seul attribut (poids) par arête.*

2.2 Tâches

Pour l'ensemble des tâches qui suivent, on vous demande de les intégrer à la base de code existante, en essayant d'avoir une structure la meilleure possible du code. On donne quelques indications les premières tâches.

ATTENTION : Même si ce n'est pas écrit dans la suite, une tâche récurrente et essentielle est de tester (et évidemment déboguer) toutes les fonctions que vous écrivez !

2.2.1 Partie 1 : Chemin de temps minimal

Cette partie consiste à implémenter des algorithmes naïfs pour la recherche d'un chemin de temps minimal entre deux points v_s et v_t . On suivra la structure suivante :

1. On considère d'abord le cas où $f(e) = 0$ pour toute arête e . Implémenter un algorithme trouvant le chemin de temps minimal entre v_s et v_t . Analyser sa complexité.

Bonne pratique d'implémentation : créer d'abord une méthode `build_simple_graph` dans la classe `Network` qui construit un graphe de la classe `Graph` correspondant à un environnement donné. Créer ensuite une méthode `shortest_path` dans la classe `Graph` calculant le plus court chemin dans un graphe pondéré quelconque. Appeler enfin ces méthodes depuis le `main.py`.

2. On considère maintenant le cas général. Proposer une formulation équivalente du problème qui consiste à chercher un plus court chemin dans un graphe étendu. Implémenter un algorithme qui résout ce problème en commençant par instancier le graphe étendu dans la classe `Graph`. Analyser sa complexité.

Bonne pratique d'implémentation : créer une méthode (l'appeler par exemple `build_extended_graph`) qui crée un objet de la classe `Graph` contenant le graphe étendu que vous avez défini. Il vous suffit ensuite de réutiliser la méthode `shortest_path` précédente.

3. Améliorer la solution précédente en ne créant que la partie nécessaire du graphe étendu.

Suggestion d'implémentation : Ici on veut pouvoir réutiliser sans modification la méthode `shortest_path` précédente sans avoir à créer toutes les arêtes du graphe. On remarque que la méthode a uniquement besoin d'avoir accès aux voisins d'un noeud à chaque fois que le noeud est visité, il faut donc simplement que la méthode `neighbours` fonctionne. Une méthode efficace est de créer une sous-classe de la classe `Graph`, appelée `GraphImplicit`, qui prend en argument non pas une liste d'arêtes mais une fonction qui peut retourner les arêtes à partir d'un noeud. Il faut ensuite s'assurer que la méthode `neighbours` fonctionne correctement. Notez qu'il est possible de le faire de façon générique (on mettra alors typiquement la sous-classe `GraphImplicit` dans le module `graph` et on mettra dans le module `network` une méthode qui construit le graphe implicite pour un réseau donné) ; ou alors de le faire de façon spécifique pour notre cas (on pourra alors mettre la sous-classe `GraphImplicit` dans le module `network` et donner à son constructeur une instance de `Network`).

En bonus, vous pouvez produire une représentation graphique (claire) du graphe et du chemin de temps minimale. Il est possible pour cela de s'appuyer sur une librairie telle que `graphviz`.

2.2.2 Partie 2 : Améliorations algorithmiques

Dans cette partie, on cherche à optimiser la performance de la recherche d'un chemin de temps minimal entre deux points v_s et v_t .

1. Proposer et implémenter une méthode dite de "pruning" qui retire les noeuds Pareto dominés du graphe étendu : si on atteint un noeud v avec une fatigue f et en un temps t alors qu'on avait déjà atteint v avec une fatigue et un temps inférieurs, alors on peut retirer l'option (v, f, t) .
2. Implémenter l'algorithme A^* avec une heuristique admissible.

2.2.3 Partie 3 : Extensions

Dans cette partie, on propose différentes extensions du problème. La première est sans doute plus facile que les suivantes. Il existe de nombreuses autres possibilités de variantes du problème et les élèves sont libres d'en choisir d'autres à condition de les étudier sérieusement. Il n'est pas nécessaire d'étudier plusieurs extensions, il est préférable d'analyser en profondeur l'une d'entre elles.

1. Cas de plusieurs missions pour un seul agent. Supposons que l'agent ait plusieurs missions à réaliser, dont l'ordre est imposé : $v_s^1 \rightarrow v_t^1, v_s^2 \rightarrow v_t^2$, etc. L'agent démarre au point v_s^1 et doit effectuer toutes les missions, dans l'ordre en allant à chaque fois du point final d'une mission au point initial de la suivante. Implémenter un algorithme trouvant les chemins minimisant le temps d'arrivée de la dernière mission. Que se passe-t-il dans le cas où on peut choisir
2. Cas de plusieurs agents, avec une mission par agent. Supposons qu'on a k agents et k missions $v_s^i \rightarrow v_t^i$ pour $i = 1..k$. Les agents sont initialement positionnés arbitrairement dans le réseau : l'agent a_j se trouve au point $v_0^{a_j}$. On doit assigner à chaque agent exactement une mission et si on assigne l'agent a_j à la mission i le cout total en temps consiste en le temps pour que l'agent rejoigne le point initial de la mission plus le temps de la mission. Trouver l'appariement permettant de minimiser la somme des couts en temps de chaque missions.
3. Points de repos. Dans le cas d'un seul agent : supposons qu'on dispose d'un lieu de repos, permettant de réduire la fatigue. Trouver un algorithme permettant de déterminer l'endroit optimal où placer ce lieu de repos dans le graphe.
4. Version multi-agent : proposer une version multi-joueur du jeu (en commençant par 2 joueurs).

Vous pouvez aussi faire une représentation interactive du jeu, par exemple avec la library `pygame`. Attention toutefois, cela nécessite l'ouvrir d'une nouvelle fenêtre et n'est donc pas possible sur Onyxia. Vous devrez travailler soit en local sur les machines de salles de TP, soit sur vos machines personnelles. Si vous ne souhaitez pas le faire, il est tout à fait possible de faire un projet sans aucune représentation interactive nécessitant l'ouverture d'une nouvelle fenêtre.

2.3 Organisation indicative des séances

L'organisation des séances ci-dessous est indicative et a pour but de vous donner une idée, elle n'est pas contraignante et pourra être adaptée par les chargés de TP en fonction des groupes.

Note : il s'agit de séances de 1h30, et comme indiqué au début du document, cela correspond à la partie accélérée du projet (c'est précédé de 4 autres séances pour ceux qui ne sont pas dans le cours accéléré).

- Séance 1 : Prise en main de l'environnement et création d'un dépôt git, prise en main de la base de code existante.
- Séance 2 : Partie 1.1
- Séances 3-4 : Parties 1.2 et 1.3. Au début de la séance 3, le chargé de TP fera un point sur le graphe étendu à considérer.
- Séances 5-6 : Partie 2. Au début de la séance 5, le chargé de TP fera un point sur les possibles optimisations (pruning et A*).
- Séance 7-9 : Partie 3. Au début de la séance 7, le chargé de TP fera un point sur les pistes possibles pour les différentes extensions, notamment la 2.

La séance 10, qui est après la deadline pour le rendu final, sera consacré à l'évaluation. Les enseignants auront lu les projets et passeront dans les groupes pour poser des questions. Cela sera aussi l'occasion de donner un retour aux étudiants sur leurs projets.

2.4 Description du code fourni

Rappel : Toutes les ressources associées au projet pour les étudiants (base de code, fichiers d'input, tests préliminaires, etc.) se trouve sur le dépôt github (publique) suivant :

<https://github.com/patrickloiseau/ensae-prog26>.

Dans cette section, on décrit brièvement la base de code fournie. On rappelle toutefois qu'il est bien souvent utile de consulter le code lui-même (et sa documentation en commentaire) pour en comprendre les détails.

On fourni deux bases de modules : **network** et **graph** contenant respectivement les classes **Network** et **Graph**.

La classe **Network** définit des objets correspondant à l'environnement décrit ci-dessus. Un objet contient des **roads** qui sont les chemins entre deux points. L'attribut **roads** est un dictionnaire de la forme

```
{ v0: liste des voisins de v0,
  v1: liste des voisins de v1,
  ...
}
```

où la liste des voisins d'un noeud (**v0**) est une liste de la forme :

```
[(v1, longueur de v0>v1, coefficient de fatigue de v0>v1), (., ., .), ... ]
```

contenant tous les noeuds qui ont une arête venant du noeud (**v0**). Un objet de la classe **Network** contient également un point de départ (**start**) et un point d'arrivée (**end**) qui définissent la mission.

La classe **Graph** définit des objets de type graphe orienté. Un objet contient des **edges** qui sont les arêtes existantes entre deux points. L'attribut **edge** est un dictionnaire similaire à l'attribut **roads** de la classe **Network** mais avec un seul attribut (le poids) pour chaque arête.

Enfin le fichier **main.py** a pour but d'appeler et de faire tourner les fonctions développées dans les fichiers **network.py** et **graph.py** pour résoudre le problème.

Description des fichiers d'exemples fournis Les fichiers d'entrées (dans le répertoire **examples**) contiennent des exemples d'environnement. Ils sont de la forme suivante :

- la première contient $m \quad v_0 \quad v_k$, c'est à dire le nombre d'arêtes puis le point de départ et d'arrivée de la mission
- les m lignes suivantes contiennent chacune une arête décrite par le noeud de départ, le noeud d'arrivée, la longueur $l(e)$, et le coefficient de fatigue $f(e)$.

A noter que la liste des noeuds n'est pas explicitement définie : il s'agit simplement de la liste des noeuds rencontrés dans une arête.

Description des tests fournis Nous fournissons dans le répertoire `tests` un test préliminaire très basique. Ce test porte sur la méthode `from_file` qui est implémentée et devrait donc passer avec succès. Le but est essentiellement de vous donner un exemple de structure de test unitaire sur lequel vous pouvez vous appuyer pour développer d'autres tests unitaires. **Il est de votre responsabilité (cela fait parti du projet) d'implémenter de nombreux autres tests beaucoup plus exhaustifs tout au long du développement de votre code.** Vous devez mettre ces tests dans le repertoire `tests`.

Nous fournissons le test dans deux formats différents : `unittest` et `pytest`, vous pouvez choisir celui que vous préférez et n'en garder qu'un pour la suite.

3 Quelques rappels d'environnement

Le texte ci-dessous rappelle simplement quelques éléments permettant d'aborder efficacement le travail à faire sur le projet. Il correspond à ce qui sera abordé lors de la séance 1 par le chargé de TP.

On rappelle qu'on demande l'utilisation de vscode et qu'on recommande l'utilisation du datalab <https://datalab.sspcloud.fr/> dont une brève documentation se trouve sur pamplemousse. Pour pouvoir l'utiliser correctement, on commencera par créer un dépôt git pour chaque binôme sur github.

3.1 Créer un dépôt git sur GitHub avec les fichiers du TP

GitHub est une plateforme de dépôt de code permettant le contrôle de version, c'est-à-dire qu'on peut tracer les modifications et les mises à jour. C'est un standard en programmation, il faut savoir mettre son code sur GitHub, et prendre l'habitude de mettre à jour le dépôt quand on fait des modifications. C'est d'autant plus important quand on code sur une plateforme cloud éphémère comme la plateforme Onyxia.

La maîtrise de git ne fait pas à proprement parler partie du programme du projet de programmation en 1A, ce sera l'objet du cours de 2A et à nouveau de 3A. L'objectif ici est donc de sensibiliser les élèves à l'utilisation de git et de fournir un moyen efficace de faire le projet, sans pour autant aborder les fonctionnalités avancées. La page du cours de 3A <https://ensae-reproductibilite.github.io/website/chapters/git.html> reste toutefois une excellente ressource pour comprendre git (en sautant les parties avancées).

Pour créer votre dépôt, faites les différentes étapes ci-dessous :

Créer votre dépôt git Vous devez créer votre propre dépôt git sur GitHub **avec la même structure** que le git de base <https://github.com/patrickloiseau/ensae-prog26> (c'est à dire les même répertoire et les mêmes fichiers aux mêmes endroits).

Pour cela, une solution facile est la suivante :

- Connectez vous sur <https://github.com/> (créez d'abord un compte si vous n'en avez pas)
- Cliquez sur le "+" en haut à droite puis sur "import repository"
- Dans le champs "Your old repository's clone URL" indiquez <https://github.com/patrickloiseau/ensae-prog26>
- Dans le champs "repository name", mettez "ensae-prog26". Il est inutile de mettre votre nom dans le nom du dépôt puisque le chemin du dépôt sera déjà associé au nom de votre compte github.
- Cliquez sur "Begin Import"

Quand c'est fini, vous verrez "Importing complete! Your new repository `votre_login_github/ensae-prog26` is ready." où `votre_login_github` est votre login github et vous pourrez y accéder à l'adresse https://github.com/votre_login_github/ensae-prog26.

Cloner le dépôt git dans VSCode Maintenant le dépôt git créé ci-dessus sera le lieu sûr de stockage de la dernière version de votre code à tout moment.

Si vous utilisez la plateforme Onyxia, à chaque début de session de codage (i.e., nouveau démarrage d'un service `vs-code-python`), dans VSCode, vous devrez clone le dépôt git. Cela copiera sur le cloud le contenu du dépôt et le rendra directement accessible depuis VSCode. Pour cela, tapez "Shift+Ctrl+P" ou "Shift+Command+P" sur macOS, puis tapez ">clone". Vous verrez apparaitre l'option "Git : clone", cliquez dessus. Cela vous demande l'adresse du dépôt git, donnez l'adresse du dépôt créé ci-dessus et cliquez sur entrée. Cela vous donne enfin une liste de répertoires où mettre le clone du dépôt, choisissez "work" (ce qui mettra le clone dans `/home/onixya/work/`) puis cliquez sur OK. Ensuite, suivez les instructions pour que VSCode se connecte à votre compte github. Vous devriez arriver dans un nouvel onglet sur github jusqu'à l'étape "Congratulations, you're all set! Your device is now connected." Vous pouvez fermer l'onglet. VSCode est en train de cloner le dépôt. Lorsque que c'est fini, cliquez sur "Open". C'est terminé. Vous devriez voir dans le panneau de gauche tous les fichiers de votre dépôt avec la bonne arborescence.

Il est aussi possible de cloner le dépôt git directement à l'ouverture du service `vs-code`. Pour cela vous devez d'abord créer un token d'accès en suivant les étapes

Github > Settings > Developer Settings > Fine-grained tokens

où aller sur <https://github.com/settings/personal-access-tokens> puis

Generate new token > Only select repositories > Add permissions "Contents" read and write > Copy token

Ensuite, ouvrir le service onyxia en collant le token ainsi que l'url récupérée en allant sur le dépôt, cliquant sur **Code** et récupérant l'url http.

Note : si vous utilisez votre ordinateur personnel, vous n'avez pas à cloner le dépôt à chaque fois. Vous pouvez le faire une fois pour toute au début. Ensuite, vous pouvez simplement ouvrir dans VSCode le dossier dans lequel se trouve la copie locale du dépôt sans avoir à re-cloner.

Se familiariser avec l'utilisation de git dans VSCode Une fois le dépôt git accessible dans VSCode, il est possible de le mettre à jour directement depuis VSCode après avoir fait des modifications. La documentation officielle se trouve [ici](#), mais c'est très simple. Il suffit de cliquer sur **Commit** dans l'onglet **Source control** à gauche et de se laisser guider. Vous pouvez aussi préalablement faire "stage" de certains fichiers en cliquant sur le "+" dans l'onglet git de vscode. Essayez de faire des petites modifications et de les add/commiter (puis faire pull/push) sur le git pour vous familiariser. Notez qu'il est aussi possible d'utiliser le terminal pour faire les opérations **git add**, **git commit -m "message"** et **git push** manuellement.

ATTENTION : L'utilisation primitive de git que nous ferons dans ce projet ne dispense pas d'une bonne communication dans le binôme. En pratique, arrangez vous pour toujours faire un pull avant de modifier un fichier, et surtout pour ne jamais modifier chacun de votre côté le même fichier. Sinon vous aurez à gérer un conflit de "merge". Ce n'est pas très difficile mais avec une bonne communication vous pourrez vous en passer.

ATTENTION : On rappelle encore une fois que si vous travaillez sur SSP Cloud, il est impératif de commiter et de pusher ses modifications à la fin de chaque séance car sinon elles sont perdues.

3.2 Résumé sur l'utilisation du terminal

Le terminal est une interface en ligne de commande (par opposition aux interfaces graphiques) qui permet d'interagir avec le système d'exploitation en écrivant des instructions. Dans ce TP, vous pouvez utiliser les fonctionnalités de VSCode pour exécuter du code python (le bouton « lecture » en haut à droite) mais il peut être utile d'exécuter votre code via le terminal. Il est très pratique de connaître aussi les commandes pour se déplacer d'un répertoire à un autre ou lister les fichiers d'un répertoire.

Depuis VSCode, vous pouvez ouvrir un terminal en utilisant le menu en haut à gauche, puis en sélectionnant « Terminal » et « New Terminal ». Sur la partie inférieure de votre écran, vous obtenez l'invite de commandes qui est de la forme :

```
onyxia@vscode-python-73088-0:~/work/ensae-prog26$
```

où `~/work/ensae-prog26` est le dossier actuel. La présentation peut varier sur d'autres systèmes, mais il y a (presque) toujours au moins le nom du dossier courant.

Une première commande : pwd La commande `pwd` permet d'afficher le chemin du dossier dans lequel on se trouve. Si vous tapez la commande et validez avec la touche Entrée, vous obtenez un résultat de la forme :

```
~/work/ensae-prog26
```

Lister les fichiers du dossier La commande `ls` (pour *list*) permet d'afficher les fichiers et dossiers du dossier courant.

Changer de dossier avec cd La commande `cd` permet de changer de dossier (c'est un acronyme de *change directory*).

Le dossier parent s'écrit `..`, ainsi la commande `cd ..` permet d'aller dans le dossier `~/work/`.

Exécuter du code python La commande `python` permet d'exécuter un fichier `.py`.

```
onyxia@vscode-python-73088-0:~/work/ensae-prog26$ python mondossier/main.py
<exécution du code python>
```

où `mondossier` est le nom du dossier qui contient votre fichier `main.py` (ici par exemple `code`).

Le dossier où on exécute le code python est important : certains scripts python peuvent être exécutés dans leur dossier directement (c'est le cas de `main.py`) tandis que d'autres sont prévus pour être exécuté uniquement dans le dossier parent. C'est particulièrement important lorsqu'on utilise des chemins *relatifs*, par exemple :

```
open("examples/fichier.txt")
```

ne fonctionne que si on est dans le dossier parent qui contient le dossier `examples`.

Si à l'inverse le shell est dans le dossier `tests`, alors il faut utiliser :

```
open("../examples/fichier.txt")
```

i.e. on indique à python qu'il faut d'abord se déplacer dans le dossier parent (avec `..`) puis dans le dossier `examples` avant d'ouvrir `fichier.in`.

3.3 Classes

En Python, tout est un objet. Il existe plein de types d'objets prédéfinis, tels que les entiers, les listes, les fonctions, etc. Les classes servent à définir de nouveaux types d'objets.

Définition et instanciation de la classe Dans notre cas, on veut définir le type d'objet correspondant à un environnement, dans la classe `Network` du fichier `network.py` (le plus souvent, chaque classe est dans son propre fichier). On peut alors instancier la classe, c'est-à-dire créer un objet de la classe, avec l'instruction suivante :

```
network = Network({'lozere': [('ensae', 10, 2), ('guichet', 20, 0)], 'lozere', 'ensae')
```

Dans ce cas, la variable `network` contient maintenant un objet de type `Network`. Le dictionnaire `{'lozere': [('ensae', 10, 2), ('guichet', 20, 0)]}` ainsi que les chaînes de caractère `'lozere'` et `'ensae'` sont les arguments du constructeur de la classe `Network`.

Attributs et méthodes Cet objet a des attributs auxquels on accède par la notation `.`, par exemple `network.start` pour l'attribut point de départ. Enfin, on peut définir des méthodes, c'est-à-dire des fonctions qui agissent sur un élément de la classe, auxquelles on accède aussi par la notation `.`, mais toujours avec des parenthèses. La définition des attributs et des méthodes de la classe `Network` est faite sous la ligne `class Network`. Par exemple, une méthode `is_forbidden` sera définie dans la classe de façon similaire à une fonction :

```
class Network:
    ...
    def build_simple_graph(self):
        code de la methode
```

Le premier argument de la méthode est obligatoirement `self` et représente l'objet sur lequel la méthode est appelée.

La méthode peut-être exécutée sur l'objet `grid` avec :

```
grid.is_forbidden()
```

où on n'a ici aucun argument (mais les parenthèses sont obligatoires pour indiquer que c'est une méthode et pas un argument).

Il existe également des méthodes dites de classe, qui n'agissent pas sur un objet de la classe mais sur la classe elle-même. Elles sont précédées par le décorateur `@classmethod`. C'est le cas de la méthode `from_file` qui crée un objet de type `Network` par lecture d'un fichier d'input.

```
@classmethod
def from_file(cls, filename: str):
    ...
    return cls(roads=roads, start=start, end=end)
```

Celle-ci s'appelle comme suit :

```
network = Network.from_file(file_name)
```

Il est important de savoir écrire des classes, en particulier des méthodes dans les classes. Cela a été vu au 1er semestre et doit être acquis. Toutefois, en cas d'oubli, voici un lien vers courte introduction aux classes : <https://courspython.com/classes-et-objets.html> (vous pouvez aussi simplement vous referez au cours du 1er semestre).

3.4 Tests unitaires

Les tests unitaires permettent de tester les fonctions écrites de façon automatique. On les écrits dans un répertoire séparé, en utilisant le framework `unittest` ou le framework `pytest` de Python. Une courte introduction est donnée par exemple ici <https://gayerie.dev/docs/python/python3/unittest.html> ou là <https://datascientest.com/pytest-pourquoi-et-comment-lutiliser>.

Les tests en `unittest` s'exécutent avec la commande `python` alors que les tests en `pytest` s'exécutent avec la commande `pytest`. Attention au répertoire dans lequel on les exécute. On peut également les insérer dans `vs-code` (en cliquant sur la fiole à gauche).

Dans ce projet, on fourni un premier test qui montre la structure d'un test. **ATTENTION : Il est nécessaire et très important de compléter ces tests en en écrivant d'autres pour tester toutes les fonction que vous coderez.** Notez qu'en complément des tests unitaires qui sont une façon systématisée de tester les fonctions, il est souvent très utile d'écrire dans le `main.py` des petits bouts de code qui appellent les fonction et en affiche le résultat, pour tester qu'il correspond bien à ce qu'on attend.